Comparative Analysis of Sorting Alogrithms on NVIDIA A100 GPUs: QuickSort, Bitonic Sort, Merge Sort, and Radix Sort

Group 6 Report
Low Zhan Qi, Jodius U2221134H
Yu Zheng Wen U2322264J
Zivan Soh Hung Beng U2421341H

Abstract

Sorting is a fundamental computer science problem, yet directly porting standard sequential algorithms to parallel GPU architectures often results in severe performance degradation due to heavy branching and memory overhead. This report systematically evaluates sequential CPU baselines against custom parallel GPU implementations and NVIDIA's highly optimized Thrust library. Utilizing an NVIDIA A100 GPU, we analyze the architectural mismatches of QuickSort (thread divergence), the hardware limits of Bitonic Sort (memory bandwidth saturation), and the parallelism collapse in naive Merge Sort. Finally, we examine Radix Sort, demonstrating how non-comparative, data-parallel algorithms overcome these bottlenecks to achieve peak hardware saturation.

1 Introduction

The transition from sequential CPU execution to massively parallel GPU execution requires fundamental algorithmic shifts. Traditional algorithms designed to leverage deep CPU cache hierarchies often fail on GPUs due to warp divergence and uncoalesced memory access. This report investigates four distinct algorithmic paradigms: QuickSort (branch heavy), Bitonic Sort (predictable sorting network), Merge Sort (sequential merge bottlenecks), and Radix Sort (non comparative data parallel). Performance is evaluated across varying elements up to $N = 33.5 \times 10^6$, utilizing Execution Time, Throughput, and Roofline Modeling to map software behavior against physical A100 hardware limits.

2 QuickSort: The Thread Divergence Penalty

QuickSort relies on a "Divide and Conquer" strategy, utilizing dynamic pivots to partition arrays. Its $\mathcal{O}(N \log N)$ average complexity makes it highly efficient for sequential processing.

2.1 Performance Results

Table 1 highlights the performance collapse when QuickSort is ported naively to parallel hardware.

Table 1: QuickSort Performance ($N = 2^{20}$)

Implementation	Time (ms)	Elem/s
CPU `std::sort`	72.933	1.44×10^7
Iterative Tiled GPU	9161.403	1.14×10^5
Optimized GPU (Dynamic)	411.230	2.55×10^6
Thrust Library	0.420	2.50×10^9

2.2 Architectural Analysis

The standard CPU `std::sort` completely outperformed the custom GPU implementations. Because QuickSort is a branch heavy algorithm, the CPU's deep cache hierarchy and branch predictors process it flawlessly.

Conversely, the Iterative Tiled GPU approach exposes the GPU's primary weakness: **thread divergence**. In a GPU, threads execute in warps of 32. When a dynamic pivot forces threads down divergent conditional paths, the GPU warps lose lockstep execution and are forced to serialize. This catastrophic divergence resulted in a 9.1 second execution time.

Implementing Dynamic Parallelism mitigated this by allowing the GPU to recursively launch child kernels, offloading task management to the hardware scheduler. While this yielded a $22\times$ speedup over the iterative GPU kernel, the overhead of launching thousands of child kernels prevented it from surpassing the CPU baseline.

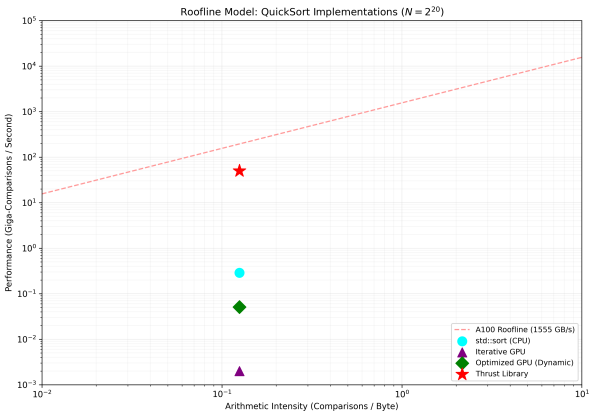

Figure 1: Roofline Model for QuickSort.

Based on the Figure 1, the QuickSort implementations are severely compute bound due to thread divergence, with all variants sharing a low fixed Arithmetic Intensity (AI) of 0.125. Even the highly optimized Thrust Library falls well short of the A100's theoretical performance ceiling, demonstrating that QuickSort's branch heavy logic cannot efficiently saturate the GPU's capabilities. All points share a fixed AI (0.125). Performance is strictly compute bound, limited by thread divergence.

3 Bitonic Sort: Hitting the Memory Wall

Unlike QuickSort, Bitonic Sort operates on a rigid, mathematically predefined network of comparisons. It is data independent, meaning its comparison paths are fixed regardless of the input data.

3.1 Performance Results

Table 2 demonstrates the extreme scaling capabilities of Bitonic Sort on GPU architectures.

Table 2: Bitonic Sort Performance ($N = 2^{20}$)

Implementation	**Time (ms)**	**Elem/s**
CPU Baseline	1,379,870	7.59×10^2
Naive GPU	1.578	6.64×10^8
Tiled GPU (Shared Mem)	1.311	8.00×10^8
Warp Shuffle	1.402	7.48×10^8
Thrust Library	0.427	2.46×10^9

3.2 Architectural Analysis

The sequential CPU completely collapsed under Bitonic Sort. The $\mathcal{O}(N \log^2 N)$ complexity requires exponentially more mathematical comparisons than QuickSort, overwhelming a single core and resulting in a 23 minute execution time.

However, moving to the GPU unlocked massive performance. Because Bitonic Sort is entirely branchless, there is zero thread divergence. The naive GPU kernel immediately dropped execution time to 1.578 ms as thousands of cores fired in perfect synchronization.

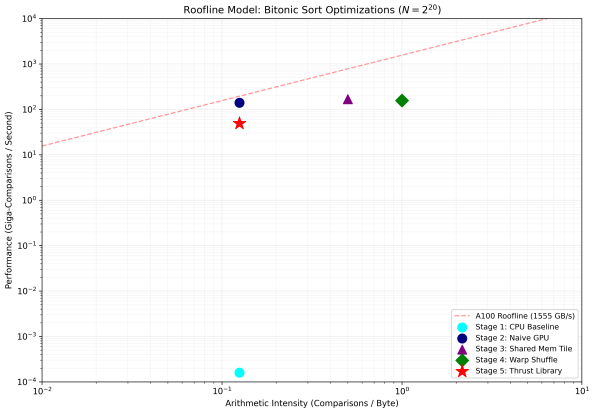

Figure 2: Roofline Model for Bitonic Sort. Hardware optimizations successfully shift AI rightward, saturating the physical memory bandwidth.

Deep hardware optimizations pushed the hardware to its absolute limits. By utilizing Shared Memory Tiling, operations were pulled into the ultra fast L1 cache, bypassing global VRAM latency and dropping the time to 1.311 ms. The Warp Shuffle implementation which allows threads to exchange data directly via internal hardware registers clocked in at 1.402 ms. The slight deceleration in the Warp Shuffle kernel is attributed to **register pressure**; at massive array sizes, keeping all data inside registers causes the GPU to "spill" overflow data back into local memory.

Ultimately, both optimized kernels clustered around the 1.3 to 1.4 ms boundary, representing the physical memory bandwidth wall of the A100 GPU for this specific comparative workload.

4 Merge Sort: Overcoming Sequential Bottlenecks

Merge sort presents a unique challenge for GPUs: while the initial divide phase is highly parallelizable, the final merge passes become increasingly sequential.

4.1 Performance Results

Table 3 details execution times across varying implementation strategies up to $N = 2^{25}$ elements.

Table 3: Merge Sort Execution Time (ms)

Algorithm	2^{10}	2^{15}	2^{20}	2^{25}
CPU `std::sort`	0.026	1.710	73.106	2470.760
CPU Merge Sort	0.061	2.467	103.106	3962.277
GPU Naive	0.389	10.814	319.138	11436.662
GPU SMEM	0.246	10.478	266.161	11376.401
GPU BSearch	0.124	0.171	0.812	32.465
GPU Thrust	0.085	0.158	0.335	2.868

4.2 Architectural Analysis

4.2.1 Small N Regime ($N \leq 2^{15}$)

At $N = 2^{10}$, all GPU variants, including Thrust, perform worse than the CPU `std::sort` (0.085 ms vs. 0.026 ms). This highlights the impact of CUDA kernel launch overhead ($\approx 5 - 10$ μs per launch). GPUs are throughput oriented devices and require sufficiently large datasets to amortize these fixed launch costs. The GPU CPU crossover point occurs at $N = 2^{15}$, where the BSearch and Thrust variants achieve approximately a $15\times$ speedup over the CPU.

4.2.2 Parallelism Collapse in GPU Naive

As visualized in Figure 3, the naive GPU implementations experience a severe parallelism collapse. At $N = 2^{25}$, the naive GPU implementation is significantly slower than the CPU baseline ($0.35\times$ speedup relative to CPU Merge Sort). In the initial passes (e.g., width=1), 16.8 million threads operate in parallel, utilizing 932 GB/s of memory bandwidth. However, during the final pass (width=$16,777,216$), exactly 1 thread merges the entire 33M element array sequentially, reducing bandwidth to 0.067 GB/s. Consequently, the final three passes consume over 70% of the total execution time, leaving the A100's SMs mostly idle.

4.2.3 Inefficacy of Standard SMEM Tiling

The GPU SMEM variant sorts 512 element tiles entirely in on chip shared memory, successfully eliminating the first 9 global memory passes. Despite this, the performance gain at $N = 2^{25}$ is negligible (11,376 ms vs. 11,437 ms). This occurs because the eliminated passes were already the fastest, accounting for only ≈ 25 ms of the total runtime. The true bottlenecks the massive, nearly sequential final merges (widths 512K to 16M) remain entirely unoptimized.

4.2.4 GPU BSearch: Parallel Co rank Merge

The BSearch algorithm fundamentally resolves the sequential bottleneck, resulting in the flattened scaling curve seen in Figure 3 (orange line). Instead of a single thread scanning $2 \times$ width elements to produce a merged output, all N threads execute in parallel. Each thread utilizes an $\mathcal{O}(\log \text{width})$ binary search to independently locate its single output element position.

This approach virtually eliminates pass to pass variance. Phase 2 passes take only 0.97 2.31 ms regardless of width, compared to the Naive variant's range of 0.29 ms to 3,987 ms. At $N = 2^{25}$, BSearch completes the phase in 32.5 ms, achieving a $351\times$ improvement over the Naive implementation and a $122\times$ speedup over the CPU merge sort, reaching a throughput of 1,034 Melem/s.

4.3 Merge Sort: Speedup & Roofline Analysis

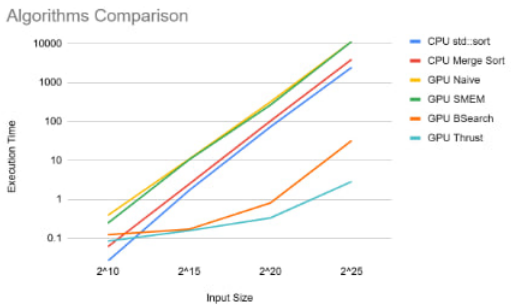

Figure 3: Execution Time vs. Input Size (N) for Merge Sort. Note the exponential parallelism collapse of GPU Naive and SMEM variants at large array sizes.

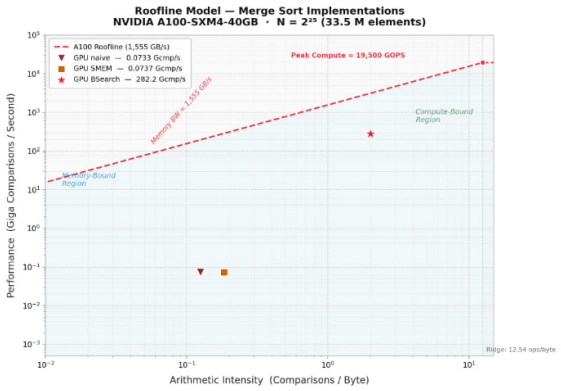

Figure 4: Roofline Model — Merge Sort Implementations. NVIDIA A100 SXM4 40GB · $N = 2^{25}$ (33.5 M elements).

All 3 GPU implementations are memory bound. GPU Naive and GPU SMEM sit at nearly the same position, since SMEM only optimizes the section that was already fast. GPU BSearch achieves much better performance than the other 2, being much closer to the roofline ceiling, and closer to the ridge point due to higher work per element. However, the roofline model alone cannot gauge real world sorting speed: a higher arithmetic intensity and better ceiling utilisation means the algorithm is doing more work per byte transferred, not necessarily finishing sooner.

5 Radix Sort: Parallelization and GPU Optimization

5.1 Introduction

LSD radix sort partitions elements digit by digit across d passes, giving $\mathcal{O}(d \cdot N)$ time. Since each pass independently assigns every element to a bucket, the algorithm is naturally data parallel and a strong candidate for GPU

acceleration. This section evaluates six implementations from a sequential CPU baseline through three progressively optimized CUDA kernels to NVIDIA's Thrust library measuring exe execution time and memory bandwidth across dataset sizes $N = 1,024$ to 33,554,432 on the Aspire2A GPU (peak bandwidth: 1,555 GB/s; peak INT32: 19,500 GOps/s).

5.2 Implementations

Table 4: Radix Sort Implementations

Implementation	Description
CPU Sequential	Standard LSD radix sort. Counts digits into 10 buckets, prefix sums, then scatters values in a stable backward pass $\mathcal{O}(d \cdot N)$, fully sequential.
GPU Sequential	Identical logic ported to device code but single threaded on GPU. Baseline exposing kernel launch overhead with zero parallelism.
GPU Naive (P1)	Full parallel decomposition: predicate kernel + manual exclusive scan + scatter kernel, repeated per digit bucket. 30 global memory kernels per digit pass.
GPU Shared (P2)	Replaces P1's 10 predicate loop with one shared memory histogram kernel + global scan + reorder kernel per pass. Atomic adds to a 10 bin shared array cut global traffic.
GPU Coalesced (P3)	Extends P2 with an in block tile shuffle stage that reorders values into contiguous buckets before the global write, ensuring adjacent threads write to adjacent addresses.
GPU Thrust (Lib)	Used as the NVIDIA Thrust library sort. Wider 8 bit radix, warp level prefix sums, hardware tuned performance ceiling.

5.3 Results

5.3.1 Performance Summary

Table 5: Radix Sort Performance Summary

Implementation	$N = 1$K (ms)	$N = 1$M (ms)	$N = 33.5$M (ms)	Speedup @33.5M	Peak BW (GB/s)
CPU Sequential	0.025	26.4	834	1x	
GPU Sequential	2.4	1,052	37,754	0.02x	
GPU Naive (P1)	17.3	25.0	134	6.2x	8.0
GPU Shared (P2)	15.5	1.4	44.8	18.6x	23.9
GPU Coalesced (P3)	7.9	1.8	42.0	19.9x	25.6
GPU Thrust	0.039	0.36	2.5	334x	433.7

5.3.2 Figures

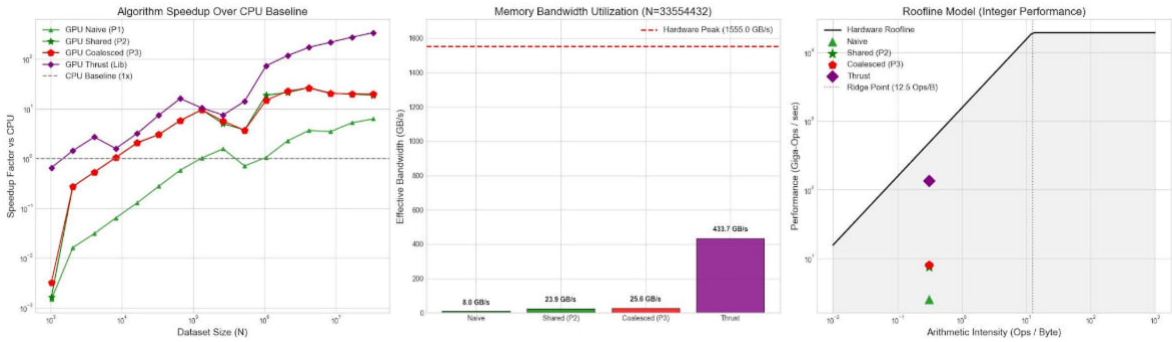

Figure 5: Left: Speedup over CPU baseline (log log). Centre: Memory bandwidth utilization at $N = 33,554,432$. Right: Roofline model (integer performance).

Figure 5 (left) shows algorithm speedup over the CPU baseline on a log log scale. Figure 5 (centre and right) are evaluated at $N = 33,554,432$ the largest test size. Figure 5 (centre) reports effective memory bandwidth against the hardware peak; Figure 5 (right) is the roofline model plotting each variant at arithmetic intensity 0.31 Ops/Byte.

5.4 Discussion

GPU Sequential. Porting counting sort unchanged to device code without parallelism incurs kernel launch overhead while retaining serial memory access, making it 40-50x slower than the CPU confirming that GPU execution does not automatically yield speedup.

GPU Naive P1. Introducing parallel predicate, scan, and scatter kernels produces the first genuine GPU speedup (6.2x over CPU at $N = 33.5$M), but 30 kernel launches per digit pass and global memory scans limit utilization to just 8.0 GB/s 0.5% of hardware peak (Figure 5, centre).

GPU Shared Memory P2. Collapsing P1's ten predicate loop into a single shared memory histogram kernel per pass is the largest single optimization step. Bandwidth triples to 23.9 GB/s and CPU speedup reaches 18.6x, reflecting the dramatic reduction in global memory traffic during counting.

GPU Coalesced P3. Adding an in block tile shuffle before the global write pushes bandwidth to 25.6 GB/s and CPU speedup to 19.9x. The marginal gain over P2 ($\sim$7%) reflects how extra shared memory synchronization barriers partially offset the coalescing benefit; P2 and P3 runtimes effectively converge at large N (Figure 5, left).

GPU Thrust. Thrust achieves 433.7 GB/s effective bandwidth 28% of hardware peak and $\sim$17x higher than P3. Its 334x CPU speedup comes from features absent in our kernels: 8 bit radix (fewer passes), warp shuffle prefix sums, and hardware specific occupancy tuning. The roofline model (Figure 5, right) confirms all implementations, including Thrust, remain memory bound at this arithmetic intensity; the performance ceiling for any radix sort at 0.31 Ops/Byte is set by the memory bandwidth slope, not compute.

6 Conclusion

This comparative analysis yields fundamental conclusions regarding algorithmic scaling on GPUs. First, brute forcing branch heavy (QuickSort) or sequentially bottlenecked (Naive Merge Sort) algorithms onto parallel hardware will inevitably fail due to thread divergence and parallelism collapse. Second, while perfectly parallel algorithms (Bitonic Sort) and advanced parallel merge techniques (BSearch) can be optimized to saturate physical memory bandwidth, their mathematical overhead sets a hard performance ceiling. Finally, the progressive optimization of Radix Sort culminating in Thrust's 2.5 ms execution time at $N = 33.5$M demonstrates that peak GPU acceleration is only achieved by transitioning to inherently non comparative, data parallel sorting networks optimized via shared memory histograms and warp level primitives.